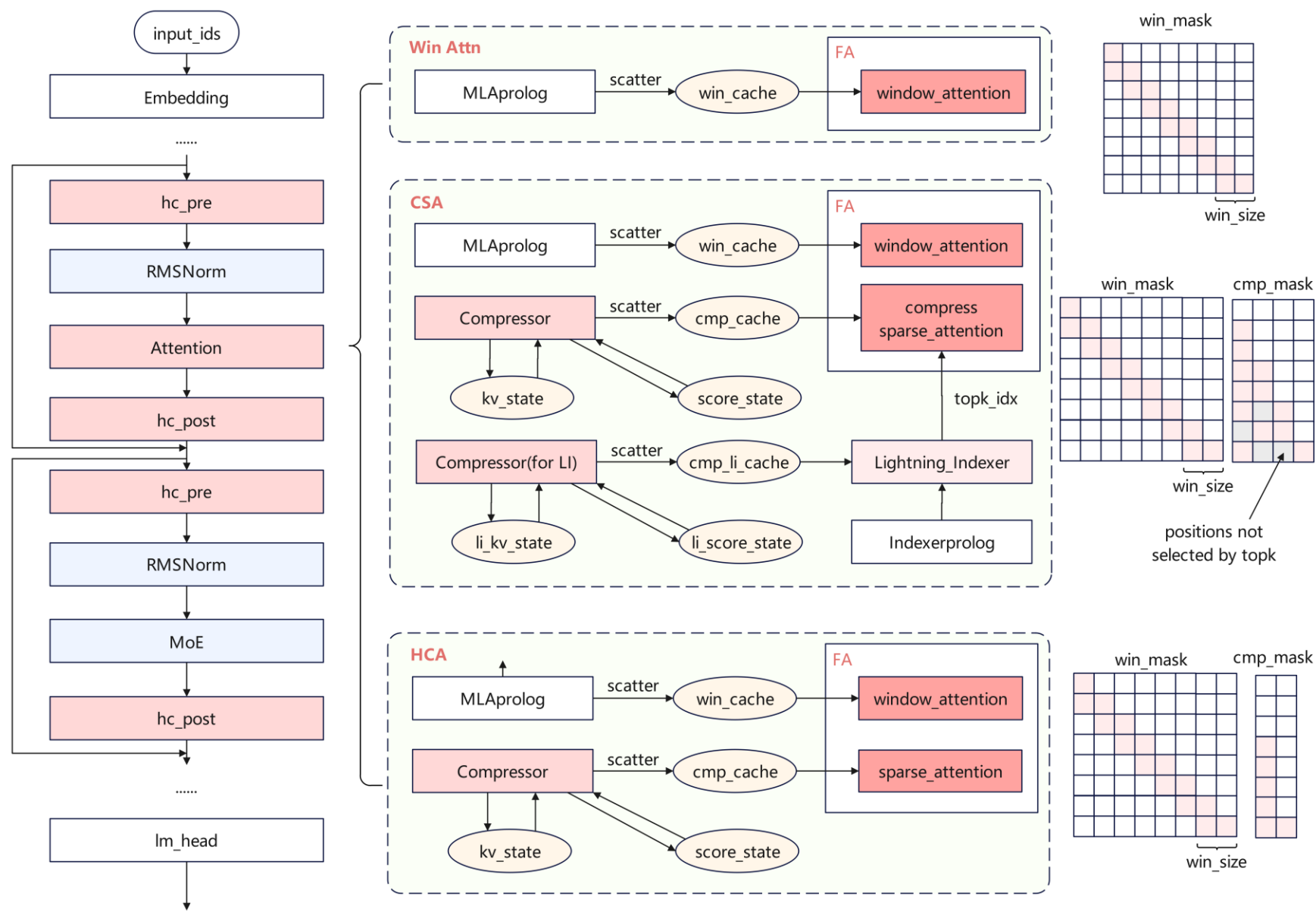


基于昇腾950的DeepSeek-V4 算子亲和优化

作者：王哲、安沙沙、谢盛伟

时间：2026/4/27

DeepSeek-V4网络结构



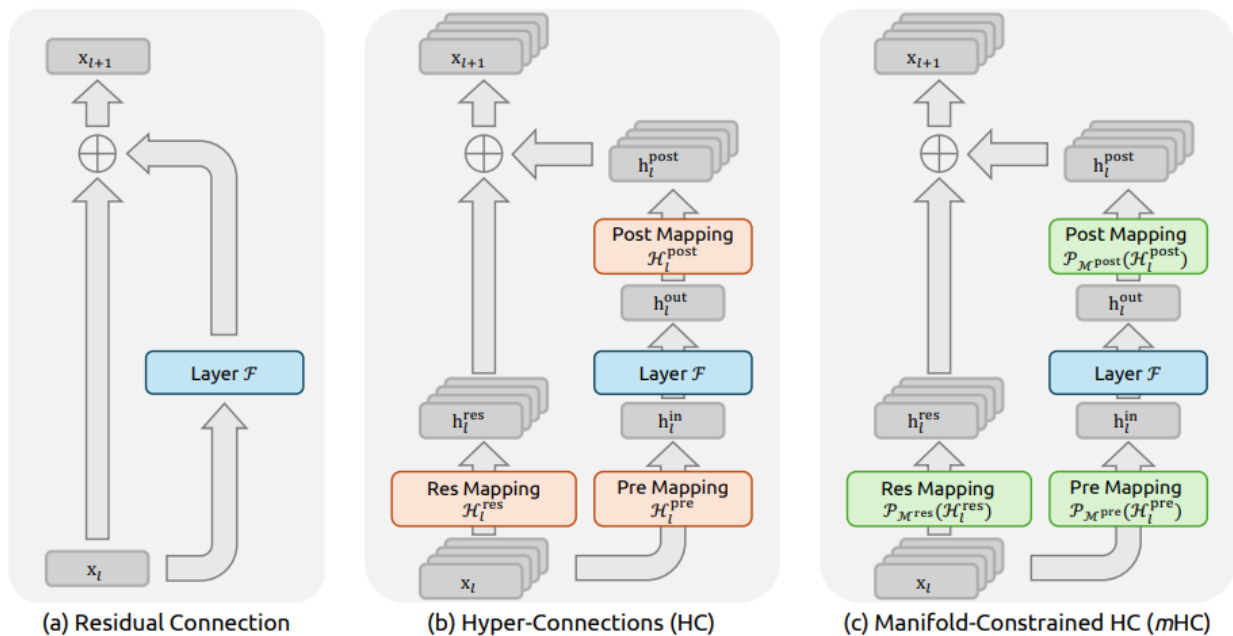
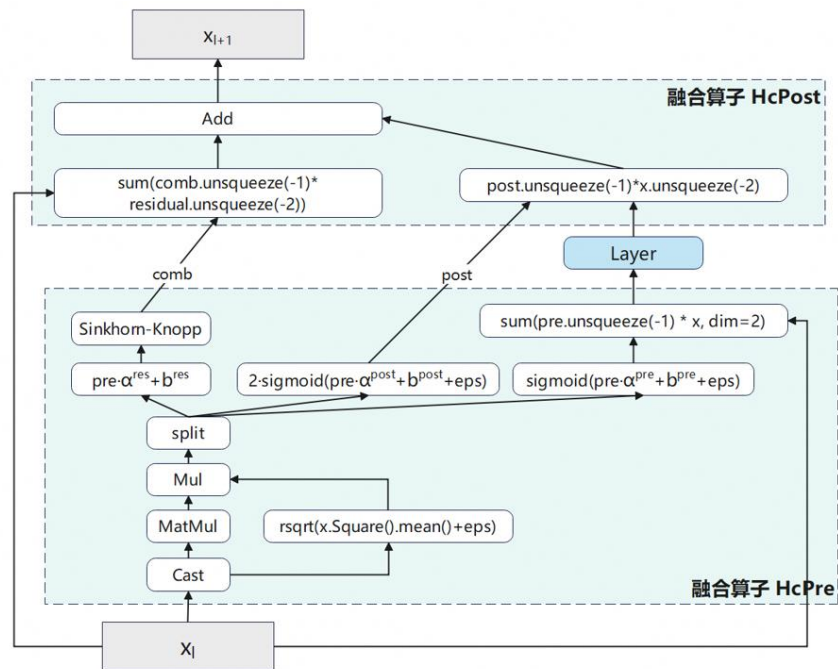


Figure source: [mHC: Manifold-Constrained Hyper-Connections](#)

➤ Manifold-Constrained Hyper Connections(mHC)

通过mHC，将residual从单流扩展到多流，拓宽了层间的信息传递效率，同时用Sinkhorn-Knopp算法应用到残差矩阵上，解决了超连接(HC) 训练不稳定等核心矛盾。



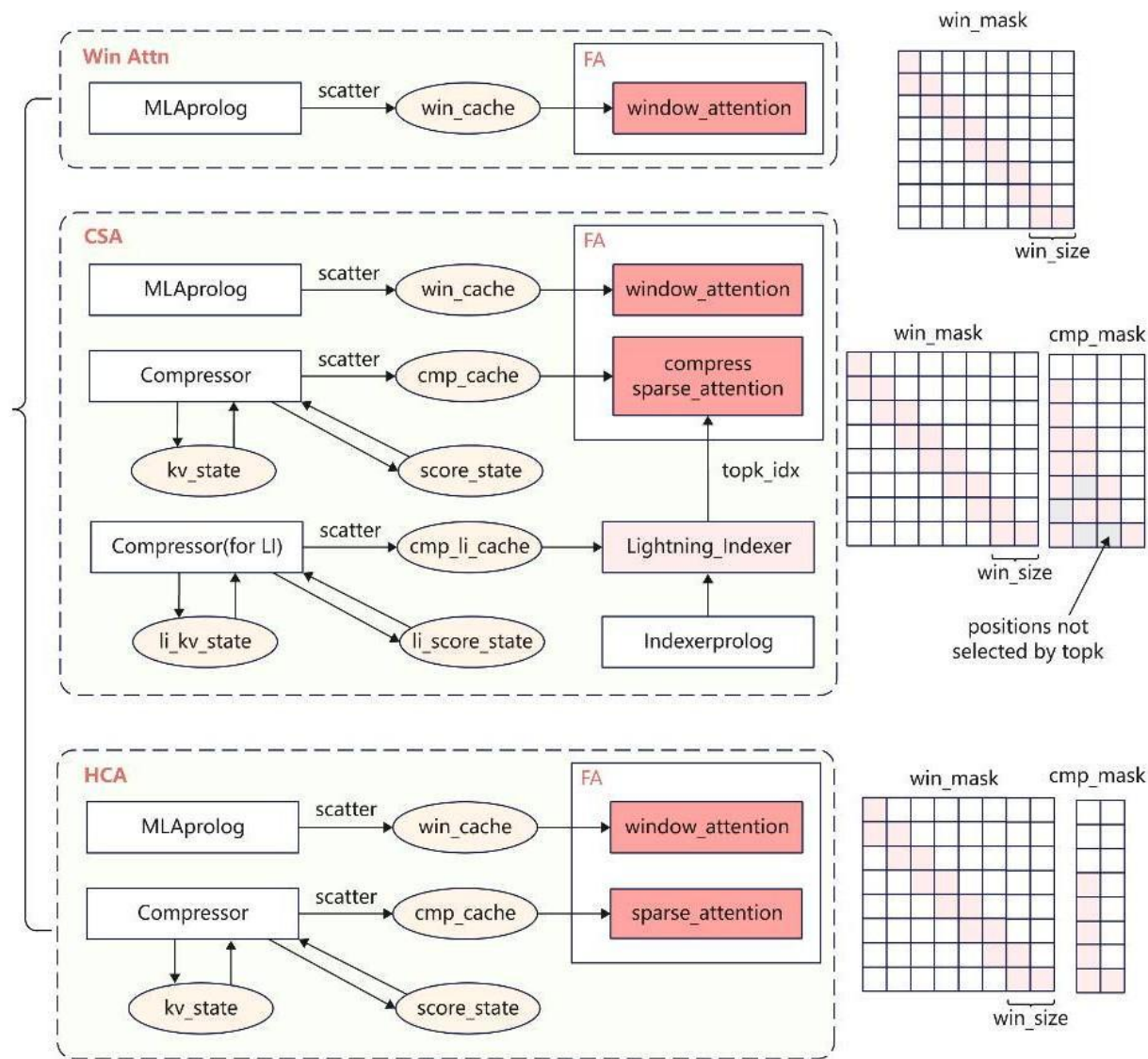
Attention

Attention 架构

➤ 多种Attention加速长序列计算

对比DeepSeek-V3.2-Exp新增了滑窗注意力机制和KV Cache压缩算法，在整网中，交织使用不同的attention结构：

- **Win Attn 滑窗注意力机制：**
 - a. 仅采用window size 128的滑窗注意力机制，不压缩KV Cache；
- **CSA 滑窗+压缩4倍+稀疏：**
 - a. 对原始上下文采用 window size 128的滑窗注意力机制；
 - b. 对原始上下文压缩4倍，压缩时存在overlap，存储压缩后的cache；
 - c. 通过Lightning Indexer稀疏选取top-K，用于DSA计算；
- **HCA 滑窗+压缩128倍：**
 - a. 对原始上下文采用 window size 128的滑窗注意力机制；
 - b. 对原始上下文压缩128倍，压缩时无overlap，不做稀疏；



Compressor

Compressor 结构

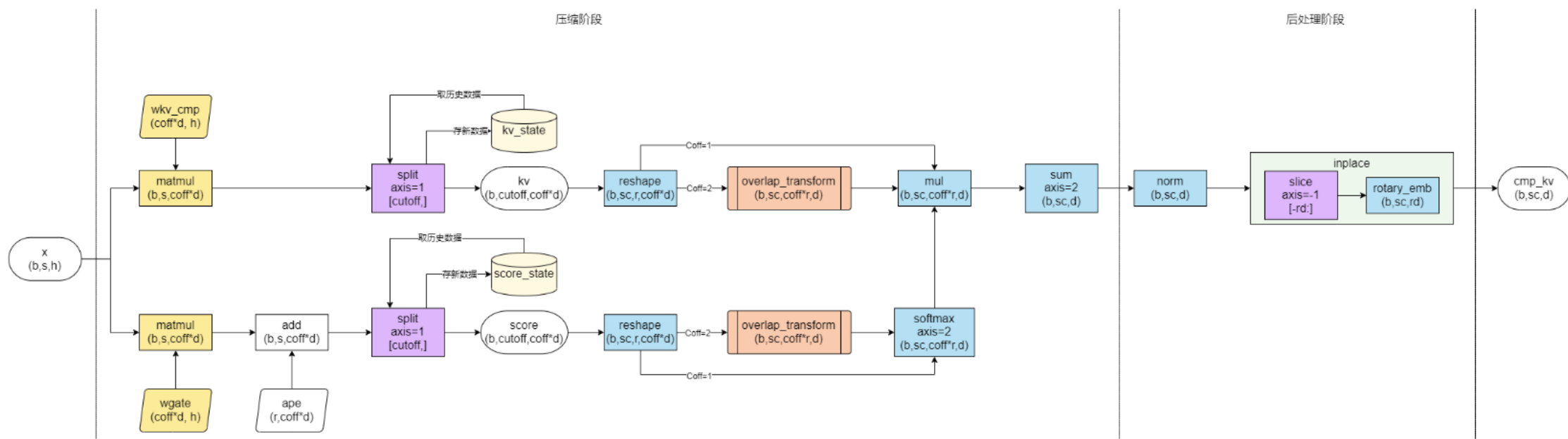
➤ 不同压缩倍率交织，大幅降低KV Cache内存占用和计算开销

• Compressed Sparse Attention(CSA) 压缩4倍，overlap token:

- 对原始上下文压缩4倍，压缩时存在overlap，每8个token压缩一次；
- 存储压缩后的 KV Cache；
- 不满足压缩长度的token信息暂存在 State Cache中。

• Heavily Compressed Attention(HCA) 压缩128倍，无overlap:

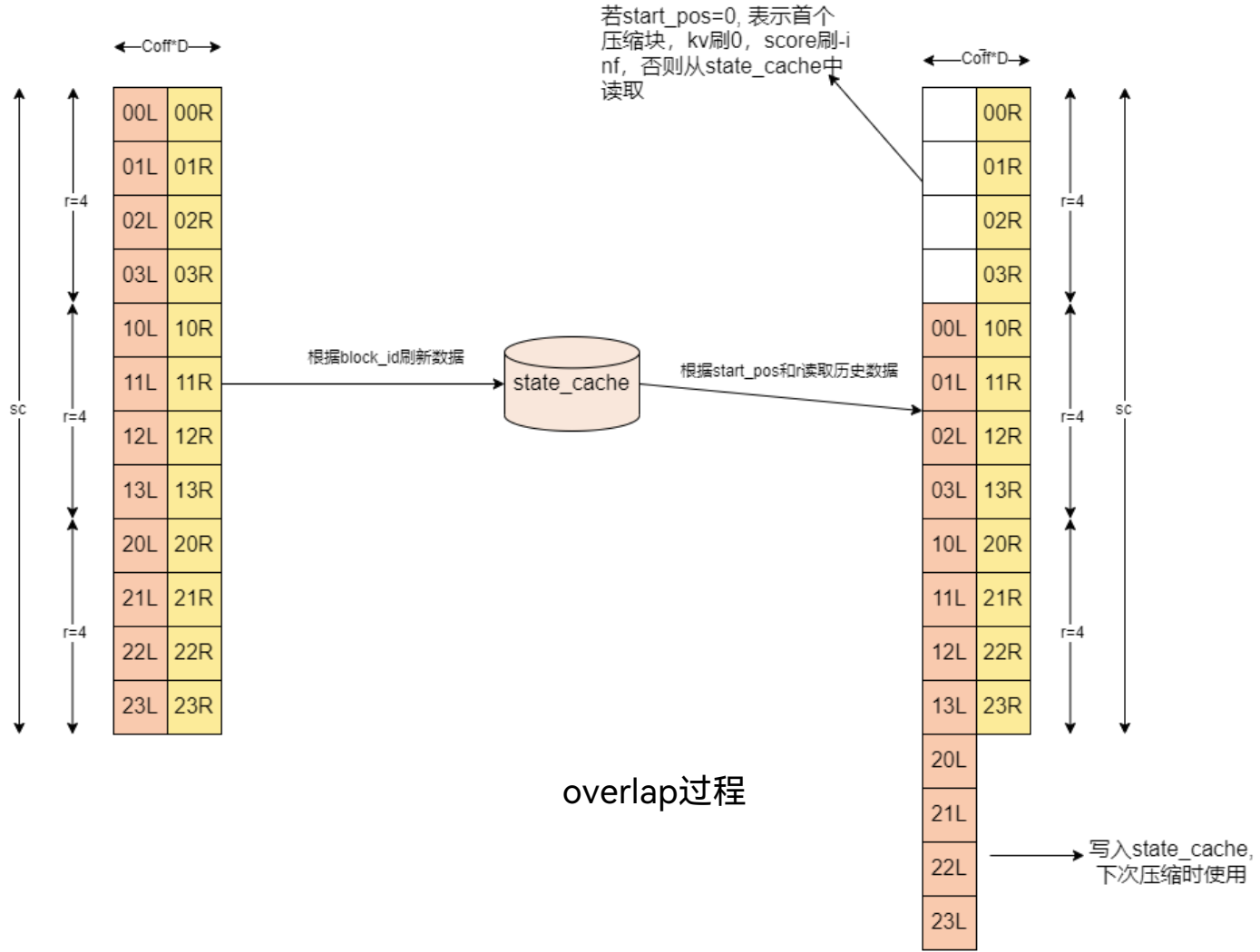
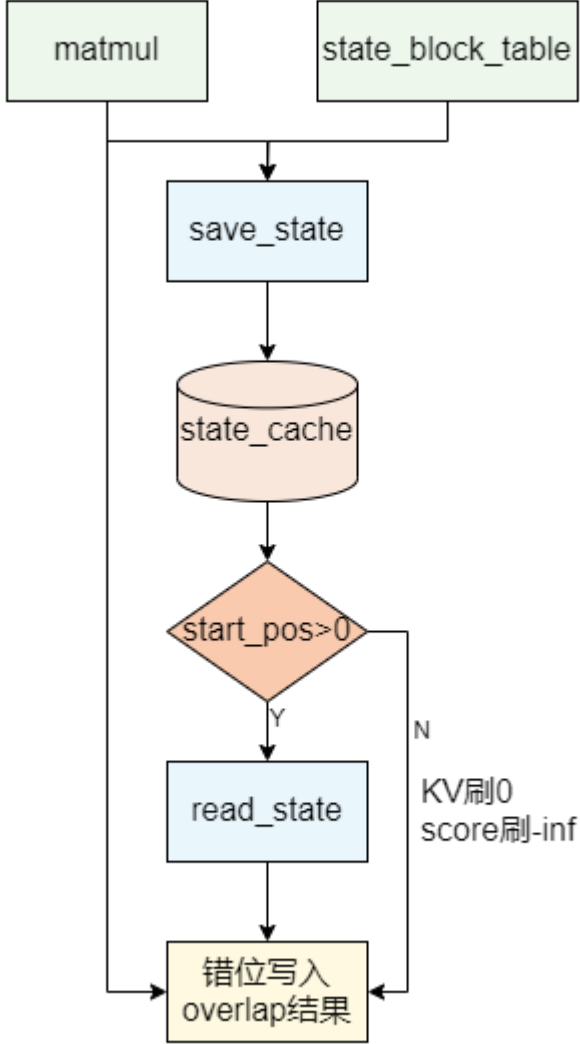
- 对原始上下文压缩128倍，每128个token压缩一次；
- 存储压缩后的 KV Cache；
- 不满足压缩长度的token信息暂存在 State Cache中。



compressor计算流程图

Compressor

Compressor 结构



Highlights

低时延&高吞吐

SparseFlashMLA

- 统一的Attention解决方案，覆盖多种Attention应用场景；
- 精细化的分核方案
- 精心设计的多级流水

LightningIndexer

- 基于直方图的桶排序TOPK算法
- 流式TOPK支持长序列

算子亲和
优化

HcPre / HcPost

- 基于Regbase编程，分别处理mHC前后处理的计算，实现完整流水线

Compressor

- 面向新架构中复杂的压缩算法，高效利用硬件带宽和算力

SparseFlashMLA算子：多场景Attention支持

算子旨在完成以下形式的 Attention 计算：

$$O = \text{softmax}(Q @ \tilde{K}^T \cdot \text{softmax_scale}) @ \tilde{V}$$

其中 $\tilde{K} = \tilde{V}$ 为基于入参控制的实际参与计算的KV，适配DeepSeek-V4模型的全新 Attention。

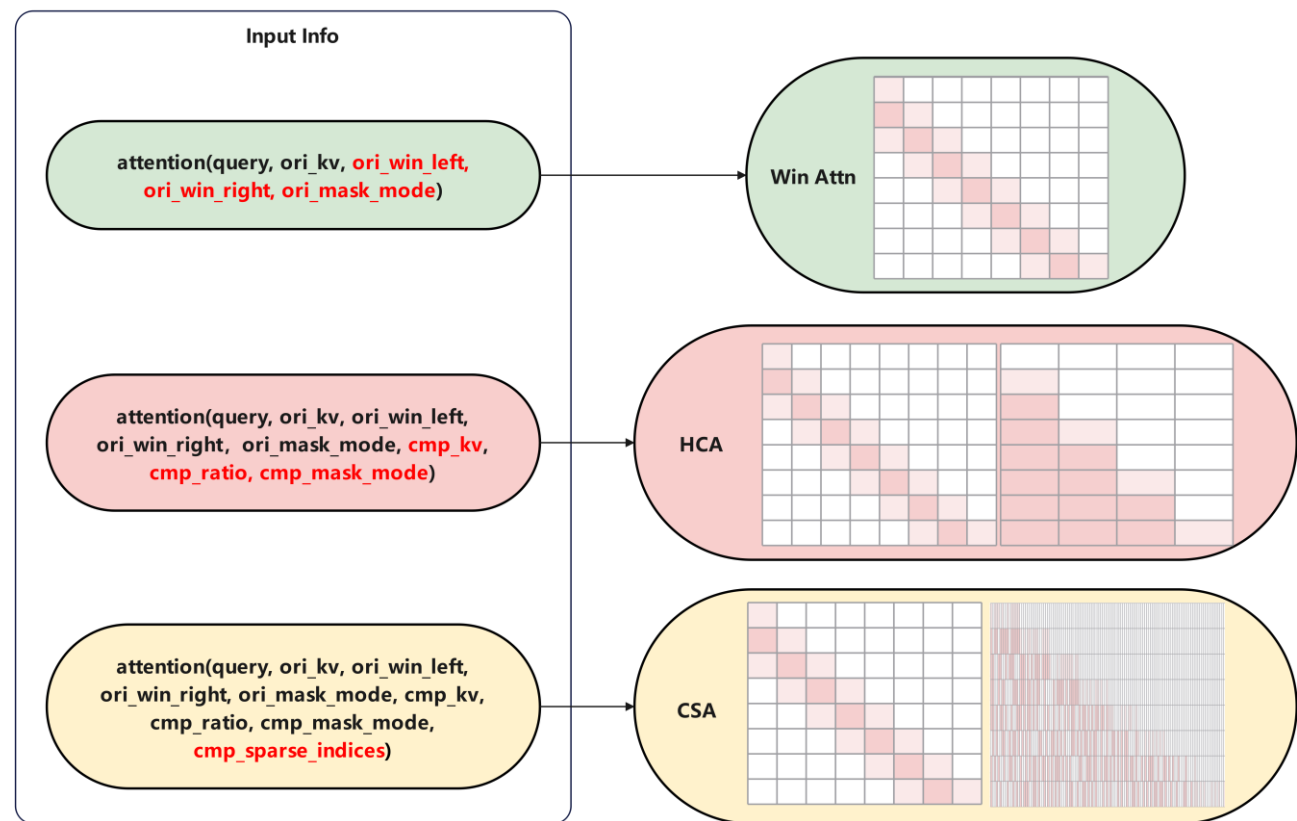
KeyPoint

- ✓ 单接口同时支持Win Attn、HCA、CSA计算。
- ✓ 同时支持稠密/稀疏计算，实现更优性能。

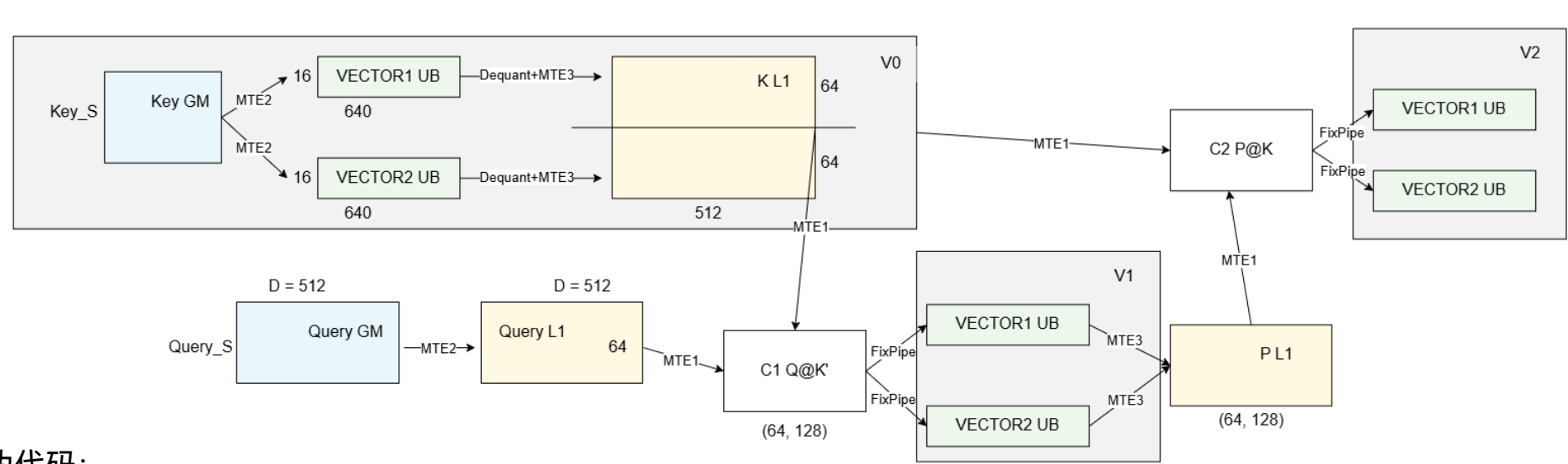
本次发布的SparseFlashMLA 算子，在 950PR 和 950DT 平台实现为KV 量化版本，即Q 支持 BF16 格式输入，KV 支持 FP8_E4M3FN 格式输入，KV 复用。

算子计算流程整体可分为五个阶段。

- V0: KeyCopyIn & Dequant(Key) -> BF16
- C1: Q@K -> FP32
- V1: online softmax -> BF16
- C2: P@V -> FP32
- V2: rescaling O -> BF16



SparseFlashMLA算子：Preload流水与同步控制



KeyPoint

✓ 支持核间/PIPELINE
间精细化同步控制，实现
多任务之间搬运、计算并
行，提高计算性能。

```
伪代码：
for i in range(n + 2):
    if i < n:
        LaunchV0;
        LaunchC1;
    if i > 0 and i < n+1:
        LaunchV1;
        LaunchC2;
    if i > 1 and i < n+2:
        Launch V2;
```

一次迭代计算的基本块大小为 (64,128)，即完成 Q 序列方向 64 和 KV 序列方向 128 长度的计算。

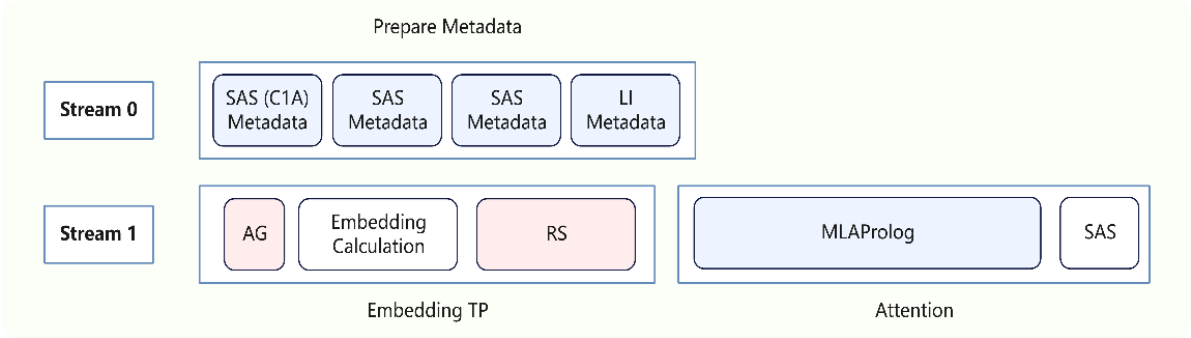
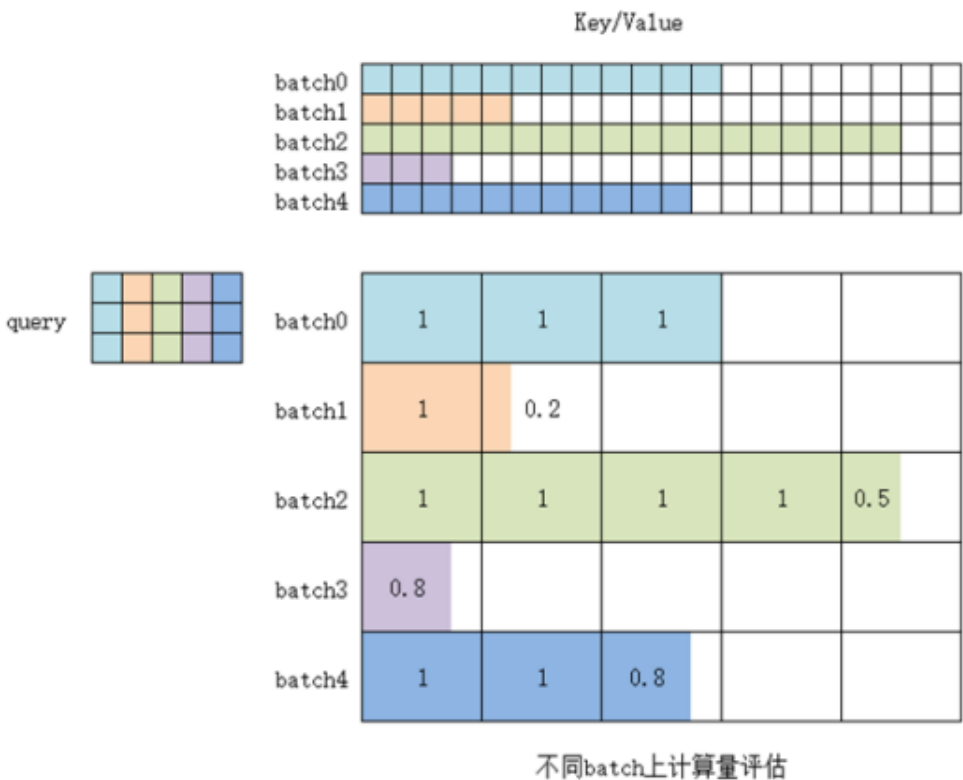
Preload流水：通过预发射掩盖Cube计算时间，达成计算bound。

Cube				C1.0		C1.1	C2.0		C1.2	C2.1			C2.2	
Vector1	V0.0			V0.1		V1.0	V0.2		V1.1	V2.0	V1.2		V2.1	V2.2
Vector2	V0.0			V0.1		V1.0	V0.2		V1.1	V2.0	V1.2		V2.1	V2.2



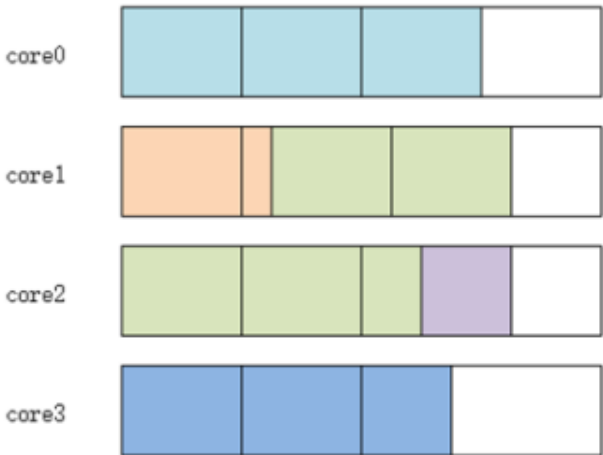
SparseFlashMLA算子：精细化分核

- 多核并行计算时，**负载最大的核决定了算子端到端计算耗时。**
- 对于FA算子，不同batch的有效token数可能不同，这一信息存在于tensor内的数据中，分核计算需要在device进行。
- 昇腾芯片内置若干个 ARM 计算核(AICPU)，把这一部分 Scheduler 计算放到 AICPU 上执行，与AICORE多流并行，掩盖计算耗时。
- 本次同步发布了Metadata AICPU算子，用于分核计算。



KeyPoint

- ✓ **AICPU算子计算分核，与AICORE计算并行，端到端掩盖耗时。**
- ✓ **负载均衡算法分核，提升整体性能。**



Compressor算子：高效带宽和算力利用

基本块选取

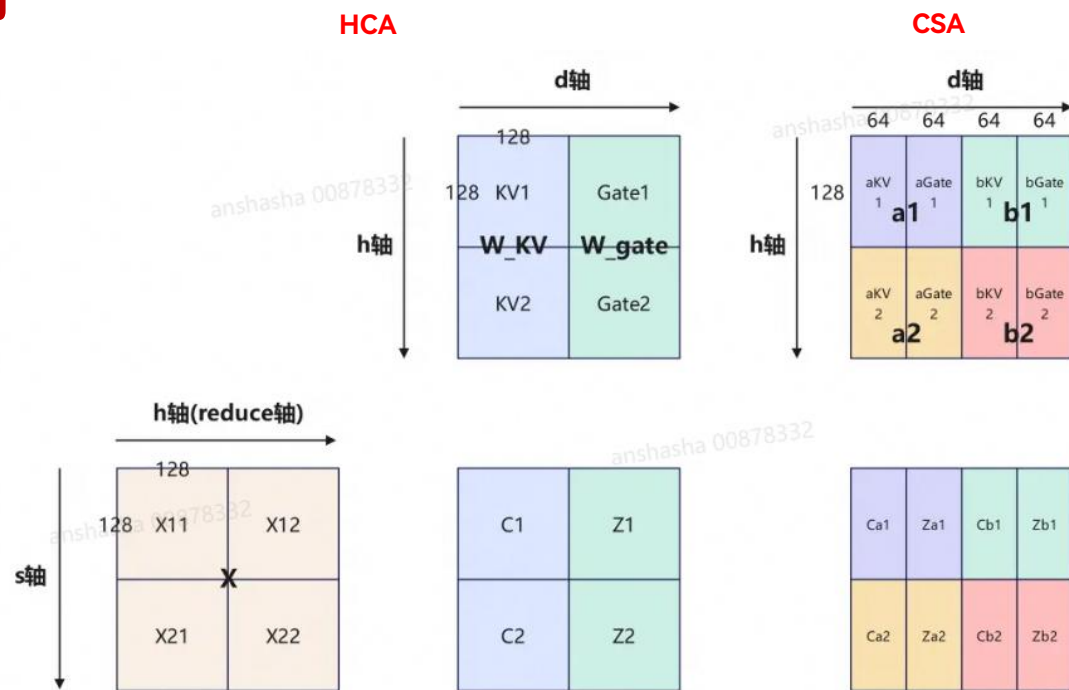
为使一轮基本块迭代中搬运左右矩阵数据的时间能被对应计算时间掩盖，需满足搬运耗时不超过计算耗时，即

$$\frac{(baseM + baseN) \times baseK \times 2 \text{ Bytes}}{\text{Bandwidth}} \leq \frac{baseM \times baseN \times baseK}{4096}$$

选取baseM=256, baseN=256, 所需带宽为64B/cycle, 结合950 GM带宽可达计算bound。

Cube内存分配：

- L1空间划分，L1基本块大小为(256,256)
 - > X矩阵使能double buffer, 分时复用：256*256*2*2 = 256KB
 - > Wkv和Wgate使能double buffer, 256*256*2*2 = 256KB
- L0A/B/C空间划分，L0基本块大小为(128,128)
 - > L0A: 使能double buffer, 2*128*128*2 = 64KB
 - > L0B: 使能double buffer, 2*128*128*2 = 64KB
 - > L0C: 分为4份，每块buffer存放(128*128) X (128*128)的矩阵乘结果，4*128*128*4 = 256KB



KeyPoint

- ✓ 在prefill场景和大batch decode场景下为计算bound
- ✓ 充分利用950 L0C内存, 算力和带宽资源, MFU可达到90%+

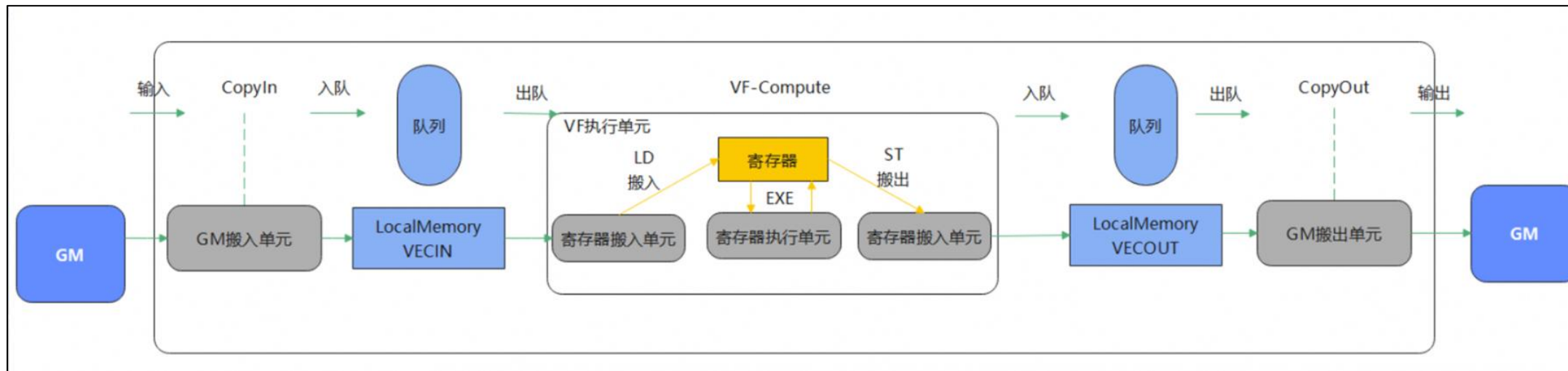
Regbase特性介绍

Regbase是950架构中新引入的与Vector计算相关的新特性。

910B/910C: Membase编程，数据运算的基本单元为LocalTensor，数据流为 GM -> UB ->GM

950: Regbase编程，数据运算的基本单元可细分到RegTensor，数据流为GM->UB->Reg->UB->GM

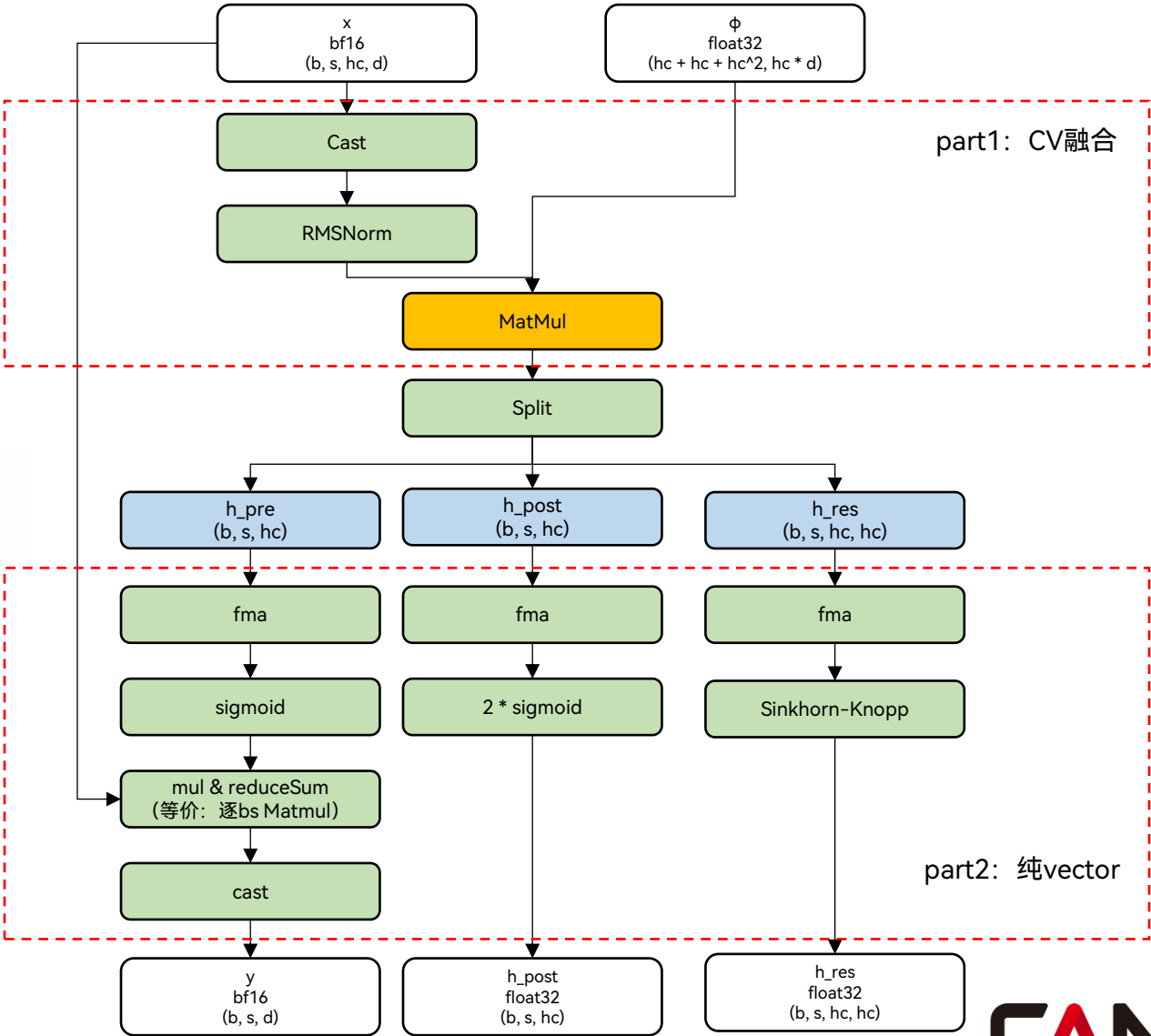
Regbase编程



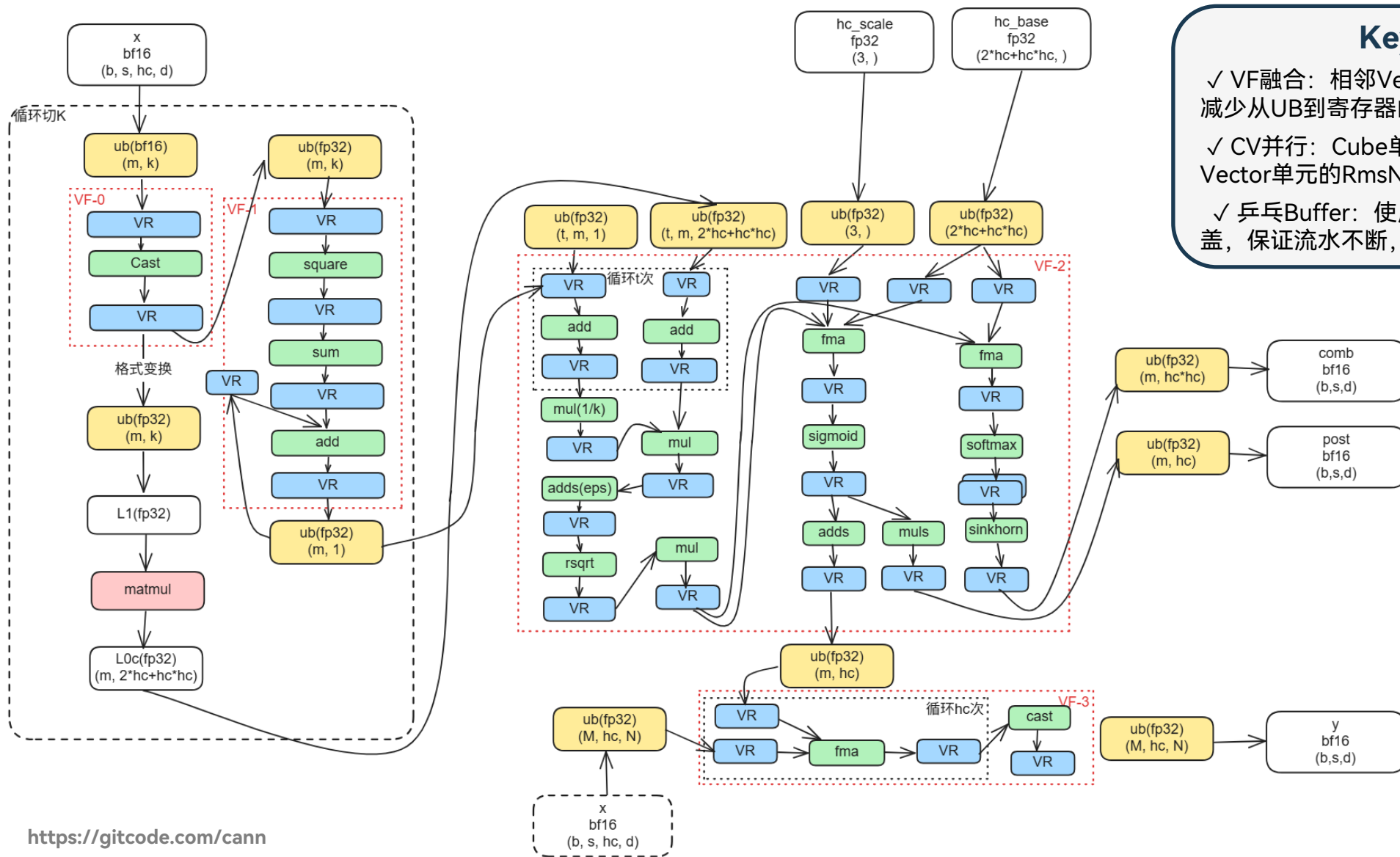
HcPre算子实现

$$\begin{cases} \vec{x}'_l = \text{RMSNorm}(\vec{x}_l) \\ \tilde{\mathcal{H}}_l^{\text{pre}} = \alpha_l^{\text{pre}} \cdot (\vec{x}'_l \phi_l^{\text{pre}}) + \mathbf{b}_l^{\text{pre}} \\ \tilde{\mathcal{H}}_l^{\text{post}} = \alpha_l^{\text{post}} \cdot (\vec{x}'_l \phi_l^{\text{post}}) + \mathbf{b}_l^{\text{post}} \\ \tilde{\mathcal{H}}_l^{\text{res}} = \alpha_l^{\text{res}} \cdot \text{mat}(\vec{x}'_l \phi_l^{\text{res}}) + \mathbf{b}_l^{\text{res}}, \end{cases}$$

$$\begin{cases} \mathcal{H}_l^{\text{pre}} = \sigma(\tilde{\mathcal{H}}_l^{\text{pre}}) \\ \mathcal{H}_l^{\text{post}} = 2\sigma(\tilde{\mathcal{H}}_l^{\text{post}}) \\ \mathcal{H}_l^{\text{res}} = \text{Sinkhorn-Knopp}(\tilde{\mathcal{H}}_l^{\text{res}}), \end{cases}$$



HcPre算子实现



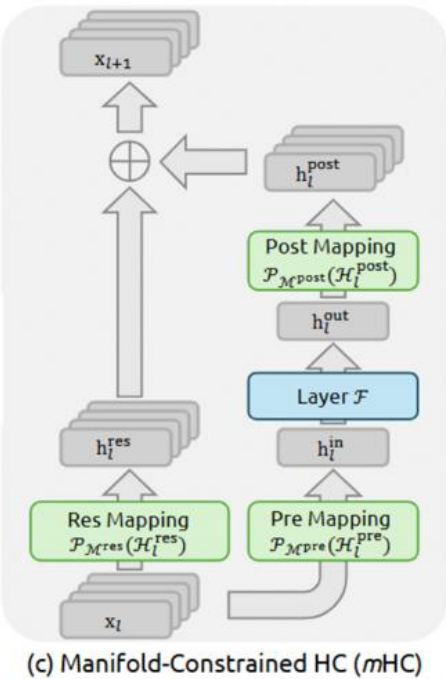
<https://gitcode.com/cann>

CANN

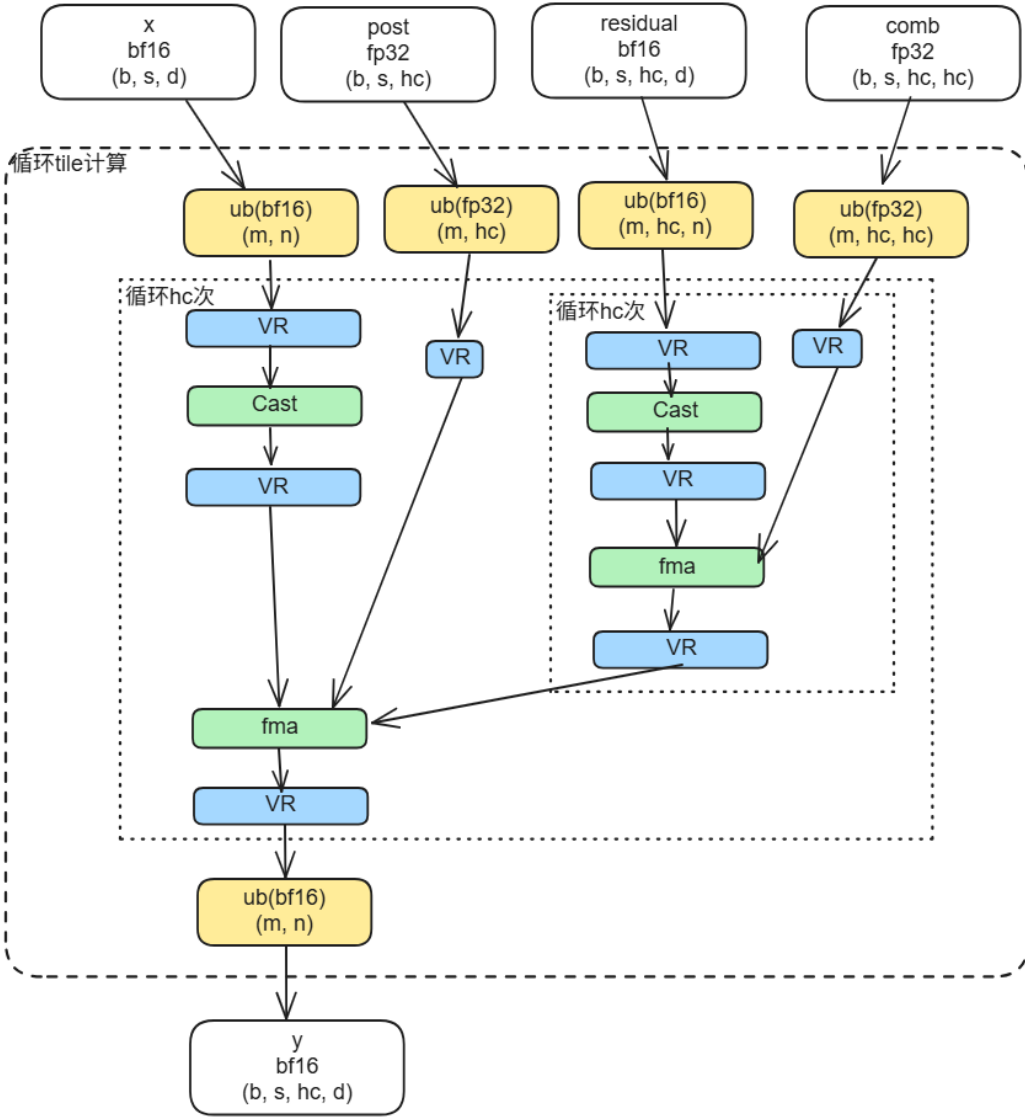
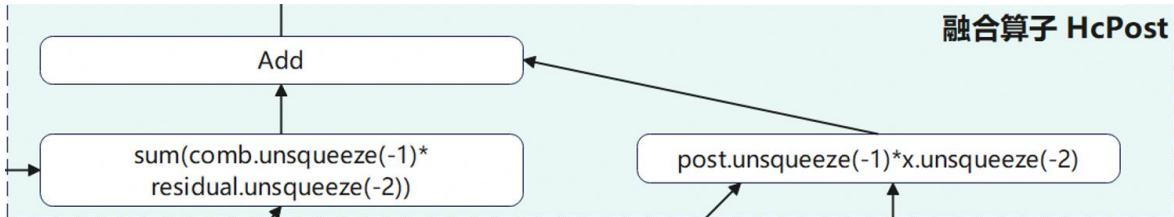
HcPost算子实现

算子功能:

- 对本层的输出x(论文中的h_out) + h_post进行Post Mapping
- 对本层的输入residua(论文中的x_l) + h_res进行ResMapping
- 对二者进行残差链接, 得到下一层的输入y



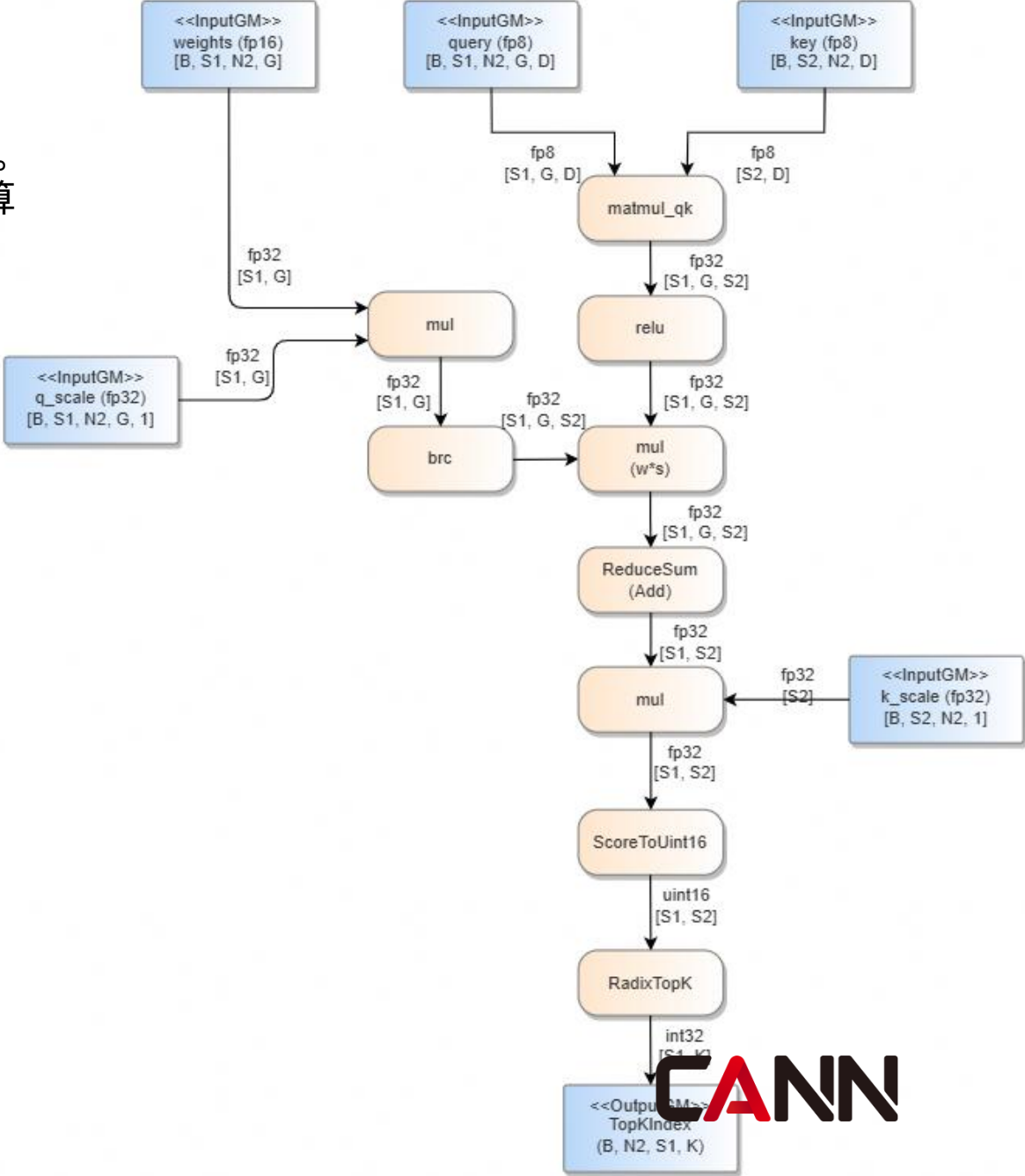
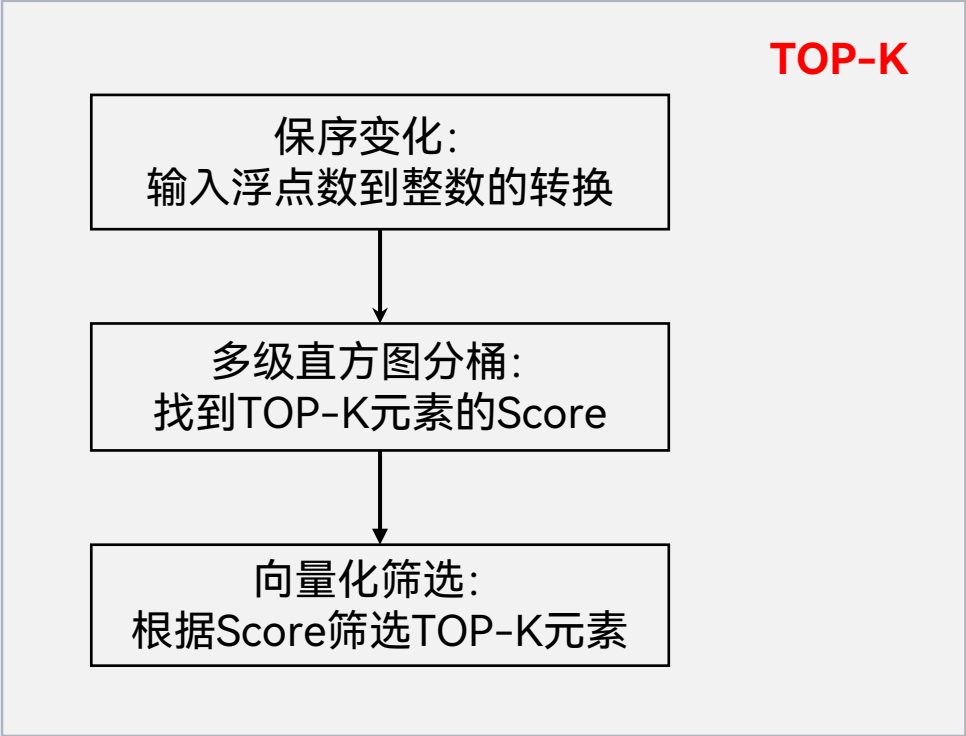
图取自论文



LightningIndexer算子

算子实现在长序列中，为每个token高效地筛选出分数最高的k个索引。同时对于算子而言，Top-k的计算必须是准确无误的，不能采用近似算法求解。

在CSA场景，KV经过压缩之后，通过LightningIndexer算子完成排序，输出排序后的index。



LightningIndexer算子：基于直方图的桶排序TOPK算法

- 指令介绍

950新增Histograms指令，可对若干个uint8_t数字进行直方图统计，一个cycle可将64个数字分到0-127或128-255桶中。

假设要在 S_k 个数中选取TOPK:

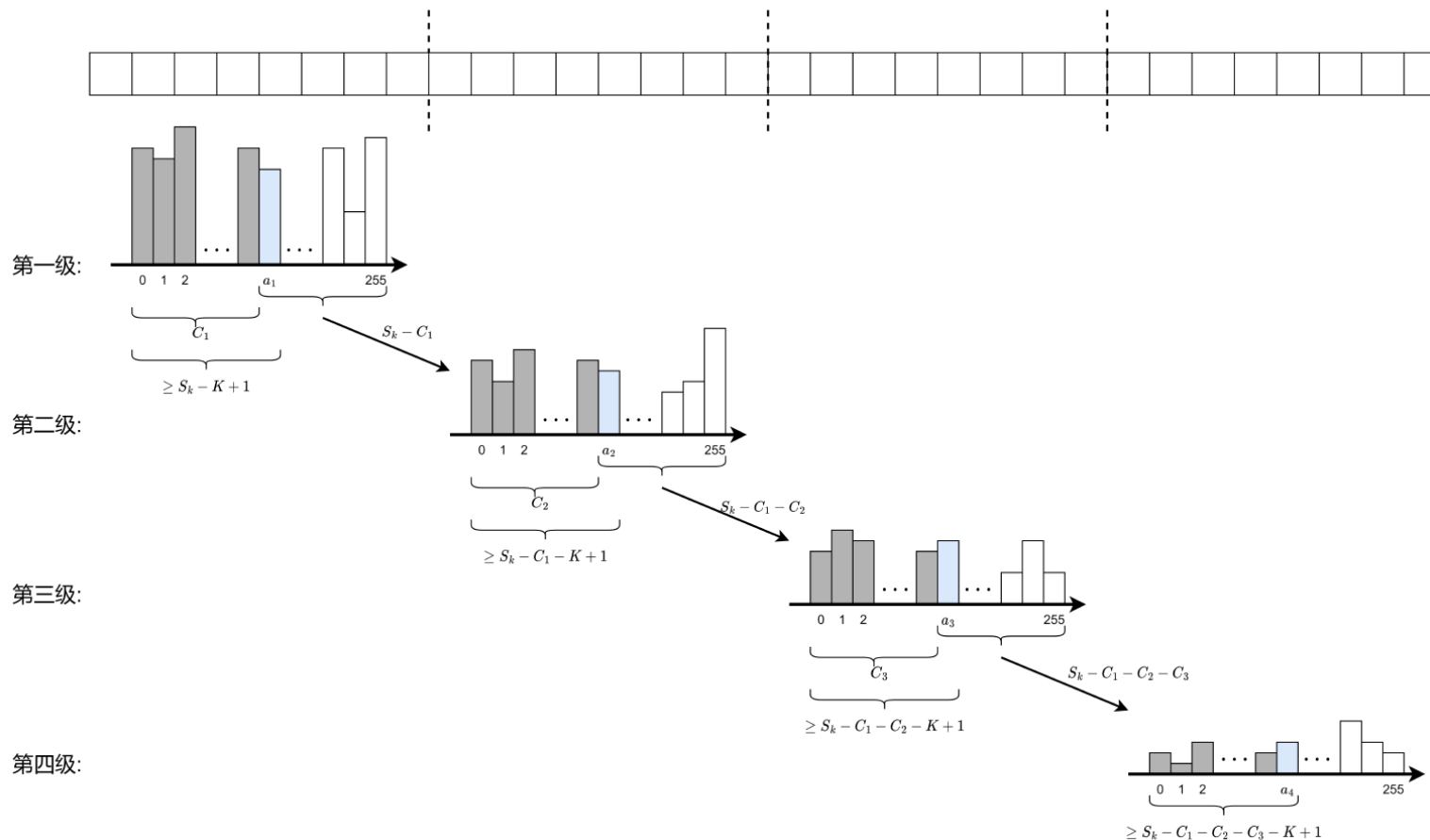
- 保序变换:

设 x 为浮点数 f 的二进制表示， $convert(f)$ 为变换后的uint32_t值，可采用如下的保序变换算法：

```
uint32_t mask = (x & 0x80000000) ? 0xffffffff  
              : 0x80000000
```

```
return (f == f) ? (xmask) : 0xffffffff
```

若输入的score是BF16类型，也可用类似方法转为uint16_t。



LightningIndexer算子：基于直方图的桶排序TOPK算法

• 多级直方图分桶

1、对所有的uint32_t元素，提取从高到低的第一个8位，用hist指令绘制直方图，返回的结果是一个长度为256的数组Array_1，其中第i个值代表0~i个桶的个数和。

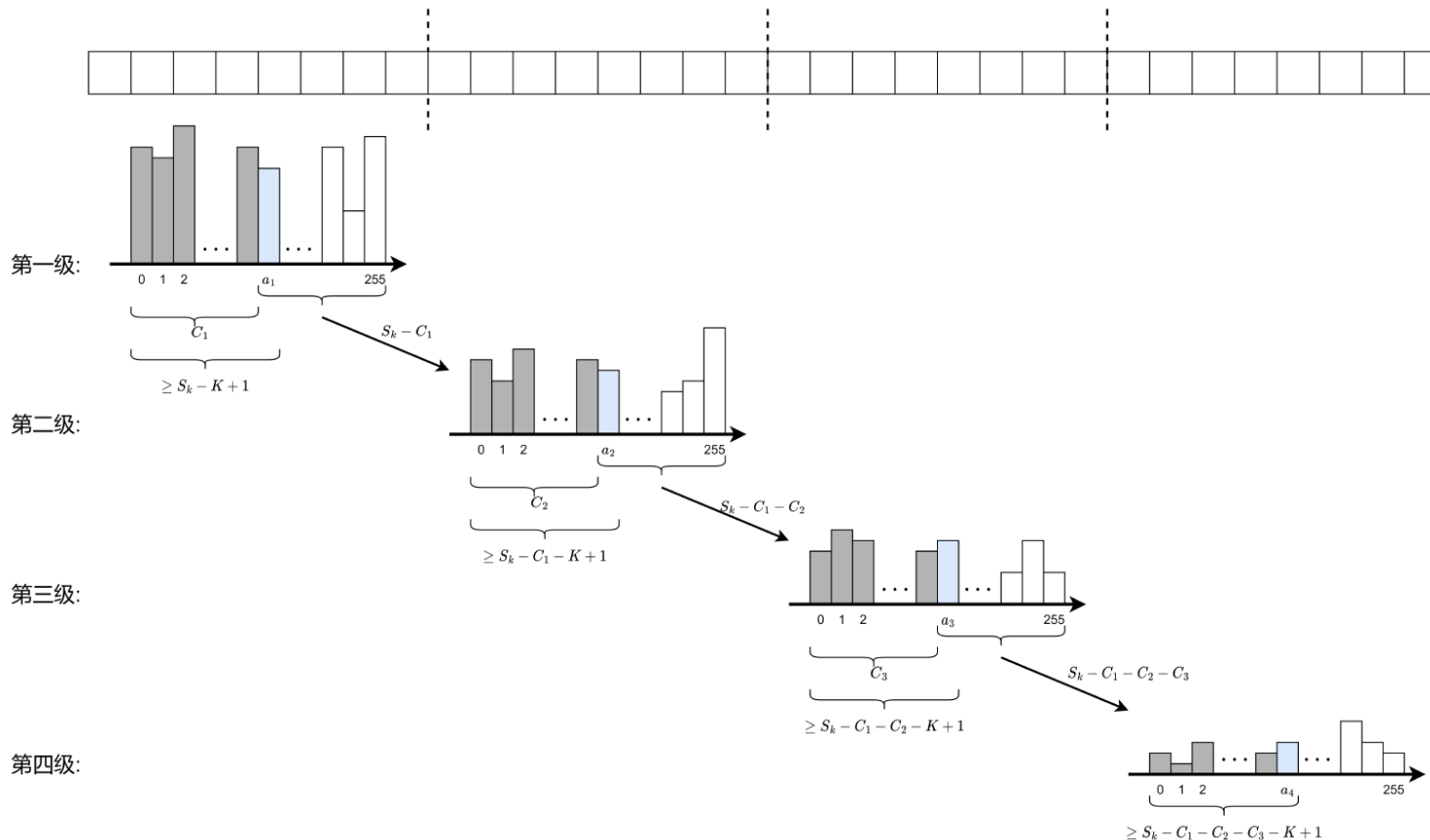
2、将Array_1中的值与临界值 $S_k - k + 1$ 比较，大于等于 $S_k - k + 1$ 的位置为true。从小到大第一个为true的桶即为第TOPK元素所在的桶(蓝色部分)，记为 a_1 。此时可将前8位小于 a_1 的元素mask掉(灰色部分)。这部分元素被丢弃，设本次丢弃的个数为 C_1 。

3、对剩余的 $S_k - C_1$ 个元素，提取第2个8位绘制直方图，重复1~2步骤，选取出临界桶 a_2 ，并丢弃第2个8位小于 a_2 的元素，统计本次丢弃个数为 C_2 。

4、重复上述过程，直到选出临界桶 a_4 。

由此得到，第k大的值为：

$$value_k = a_1 \times 2^{24} + a_2 \times 2^{16} + a_3 \times 2^8 + a_4$$



• 向量化筛选

此时已拿到第k大的值为 $value_k$ ，只需要考虑大于等于 $value_k$ 的值，流程如下：

1、大于 $value_k$ 的部分，以向量化的方式，256B为单位将原序列中的值与 $value_k$ 比较得到一个mask，大于 $value_k$ 的位置为true。以此mask筛选出所有大于等于 $value_k$ 的值对应的指标，并聚集到result中，此时个数小于k。

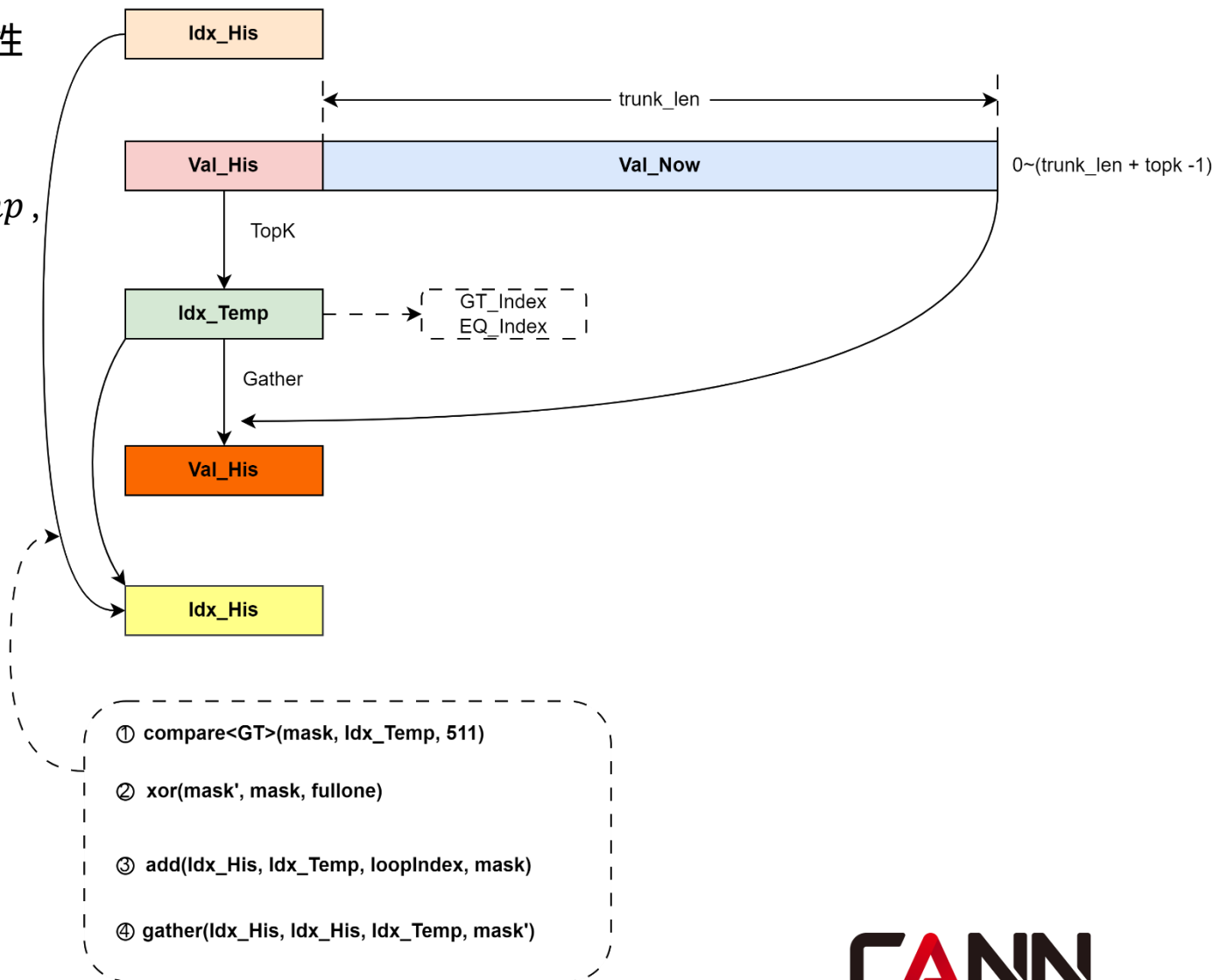
2、等于 $value_k$ 的部分，以256B为单位循环遍历所有Score，找到等于 $value_k$ 的值并筛选出来，直到result的总数等于k时退出循环。

LightningIndexer算子：流式TOPK支持长序列

- 长序列场景，受限于内存限制，不可能对完整序列一次性取出TOPK，基于流式思想分段处理。

1、首轮循环，从 $trunk_len$ 数据取出TOPK，记录 idx_temp ，并gather获取id对应的值，记为 val_his 。查找 idx_temp 在全局数据中的索引，记录为 idx_his 。

2、以 val_his ，和新的 $trunk_len$ 数据合并在一起，再次筛选TOPK，得到新的 idx_temp 和 val_his ，根据 idx_temp 计算 idx_his 。直到把长序列完整筛选，得到最终的TOPK。



亲和优化总结

SparseFlashMLA

- 多场景Attention支持
- Preload流水与同步控制
- 精细化的分核方案

Compressor

- 通过基本块选择，高效利用硬件带宽和算力

HcPre / HcPost

- 基于Regbase编程，高效利用芯片算力

LightningIndexer

- 基于昇腾950平台的TOPK算法优化
- 流式TOPK支持1M长序列

欢迎体验和贡献

CANN社区



CANN公众号



ops-transformer: <https://gitcode.com/cann/ops-transformer>

cann-recipes-infer:
<https://gitcode.com/cann/cann-recipes-infer>

欢迎广大开发者体验并参与贡献，如有疑问也可通过
issue、SIG联系我们！

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯

<https://gitcode.com/cann>

CANN